

Global placement with RePIAce

What it does Chooses approximate positions for the chip's logic cells so that connected cells are close, while cells are spread across the available space.

Remember this Connected cells pull together. Crowded regions push cells apart. Nesterov's method repeatedly updates their positions.

Where this fits in OpenROAD

After floorplanning, **global placement** distributes cells. Detailed placement then puts them on legal sites and removes overlaps. Clock tree synthesis and routing follow. Timing analysis is used repeatedly throughout the flow.

The algorithm in everyday language

Imagine arranging students in a classroom. Students doing a project together want nearby seats, but everybody cannot occupy the front row.

- 1 Start with cell positions.** Inputs include the netlist (which pins connect), cell sizes, fixed objects and usable chip area.
- 2 Estimate connection length.** Long connections receive a penalty. Moving connected cells closer usually reduces that penalty.
- 3 Measure crowding.** Divide the chip into small bins. Overfull bins create a spreading force, using an electrostatic model. These are mathematical charges, not real electrical charges.
- 4 Move cells with Nesterov updates.** Use the previous movement to form a look-ahead position, then use the slope of the cost there to choose the next move. This is accelerated gradient optimization.
- 5 Repeat and hand off.** Balance wire length against density until stopping criteria are reached. Output: approximate cell locations for legalization.

The two competing effects



Why it matters Poor placement creates long wires and congestion. Later timing repair and routing then have a harder problem to solve.

Global placement with RePIAce

One small numerical example

A and B are connected. A is at (0,0) and B at (10,0). Their simple Manhattan distance is 10 units. Moving B to (4,0) reduces it to 4 units.

But if many other cells already occupy that region, moving B there increases crowding. The placer must consider **both** effects, rather than just making every wire shorter.

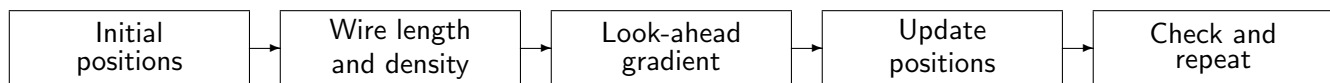
A simplified objective

$$\text{cost} = \text{smooth wire-length estimate} + \lambda \text{ density penalty}$$

Here λ controls how strongly crowding matters. OpenROAD uses a detailed numerical model; this expression explains the central idea.

Why Nesterov is useful

An ordinary gradient step looks for a downhill move from the current position. A Nesterov step uses momentum from previous movement and evaluates the direction at a look-ahead position. This can reach a useful placement with fewer iterations. It does not guarantee the best possible chip layout.



How timing and routing influence placement

Timing driven mode Gives more weight to connections with poor slack, encouraging the placer to shorten important connections. Slack means the time margin before a timing violation.

Routability driven mode Estimates routing congestion and can inflate the effective area of cells in crowded routing regions, encouraging more space there.

Tradeoff Spreading cells helps density and routing, but may lengthen wires. Pulling everything together can create overlap and congestion.

A command to recognize

```
global_placement -timing_driven -routability_driven
```

This is one stage command, not a complete script. The design, libraries, floorplan, timing constraints and wire RC model must already be set up.

What to say to your professors

“OpenROAD uses RePIAce for global placement. It treats wire length as a pulling effect and cell density as a spreading effect. Nesterov’s accelerated gradient method updates the cell positions. Detailed placement then makes those positions legal.”

Sources and code OpenROAD Global Placement documentation; Lu et al., ePlace, 2015. Implementation: `src/gpl/src/nesterovPlace.cpp` and `nesterovBase.cpp`. Examples in these handouts are illustrative, not measured OpenROAD results.